🏛 GLA UNIVERSITY, MATHURA — DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

HPC Masterclass | Day 3 In-Class Assessment: Deep Learning on HPC, oneDNN, Slurm & Horovod

Target Audience: 7th Semester B.Tech CSE (HPC Elective) | **Total Questions**: 25 | **Duration**: 30 Minutes

📋 Answer Key & Forensic Solution Guide (Instructor Copy)

Q#	Correct Answer	Cognitive Bloom Level	Core Concept Focus
01	B	`Understand`	In enterprise HPC clusters, what is the primary fu...
02	B	`Analyze`	In hardware-aware deep learning optimization, how ...
03	B	`Knowledge`	What is Intel oneDNN (formerly MKL-DNN), and what ...
04	B	`Understand`	How does oneDNN operator fusion (e.g., fusing Conv...
05	B	`Application`	Which environment variable explicitly forces moder...
06	B	`Analyze`	Why do Intel oneDNN primitives convert standard te...
07	B	`Understand`	What is Intel AMX (Advanced Matrix Extensions) int...
08	B	`Knowledge`	What architectural capability characterizes the In...
09	A	`Knowledge`	Which software package enables standard TensorFlow...
10	B	`Understand`	What is the primary compute architecture of Habana...
11	B	`Analyze`	What unique networking integration distinguishes H...
12	B	`Knowledge`	What is the role of Intel Habana SynapseAI softwar...
13	B	`Understand`	In a Slurm-managed HPC cluster, what is the role o...
14	B	`Application`	Which Slurm batch script directive specifies that ...
15	B	`Knowledge`	What does the Slurm command 'squeue' perform when ...
16	A	`Application`	Which environment variable is automatically inject...
17	B	`Analyze`	Why is the '#SBATCH --exclusive' flag frequently u...
18	B	`Understand`	How does Horovod differ fundamentally from traditi...
19	B	`Analyze`	What is the primary network scalability bottleneck...
20	A	`Apply`	In the Ring-AllReduce algorithm utilized by Horovo...
21	B	`Knowledge`	Which Horovod API function must be invoked at the ...

Q#	Correct Answer	Cognitive Bloom Level	Core Concept Focus
22	B	`Application`	In a multi-GPU/XPU server running Horovod, why is ...
23	B	`Application`	In TensorFlow 2.x with Horovod, how is automatic g...
24	B	`Understand`	Why is 'hvd.broadcast_variables(model.variables, r...
25	B	`Understand`	According to Goyal et al.'s Linear Scaling Rule in...

🔬 Complete Questions with Detailed Technical Rationale

Question 01

In enterprise HPC clusters, what is the primary fundamental difference between running machine learning workflows on a dedicated cluster versus a standard single workstation?

- **(a)** HPC clusters only run interpreted Python without native C++ compilation.
- **(b) [CORRECT]** HPC clusters decouple compute and storage across distributed nodes connected via ultra-low-latency fabrics (e.g., InfiniBand/RoCE) and leverage parallel file systems (e.g., Lustre/GPFS).
- **(c)** HPC clusters do not support GPU acceleration and rely exclusively on single-threaded CPUs.
- **(d)** HPC workstations automatically eliminate the need for batch queuing systems like Slurm.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: HPC clusters aggregate massive compute capacity across multiple distinct nodes linked by ultra-low-latency interconnects (InfiniBand or RoCE) and high-throughput parallel filesystems (Lustre/GPFS) to eliminate I/O bottlenecks during distributed training.

Question 02

In hardware-aware deep learning optimization, how are standard neural network layers fundamentally categorized regarding hardware execution constraints?

- **(a)** Static vs. Dynamic layers.
- **(b) [CORRECT]** Compute-bound layers (e.g., GEMM, Conv2D) and Memory-Bandwidth-bound layers (e.g., LayerNorm, Softmax, ReLU).
- **(c)** Single-precision layers vs. Double-precision layers only.
- **(d)** Scalar layers vs. Microcontroller layers.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Hardware-aware ML optimization categorizes operations into compute-bound (arithmetic intensity is high, e.g., matrix multiplications/convolutions) and memory-bandwidth-bound (arithmetic intensity is low, e.g., activations, normalizations, elementwise additions), dictating whether FLOPs or memory bus width limits throughput.

Question 03

What is Intel oneDNN (formerly MKL-DNN), and what role does it serve in deep learning software stacks?

- **(a)** A high-level GUI for annotating image datasets.
- **(b) [CORRECT]** An open-source, cross-platform performance library of basic deep learning primitives optimized for Intel architecture CPUs and GPUs.
- **(c)** A proprietary compiler replacing GCC and Clang for operating system kernels.
- **(d)** A specialized database engine for storing neural network weights.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: Intel oneDNN (oneAPI Deep Neural Network Library) provides highly tuned implementations of core DL building blocks (convolutions, pooling, inner products, activations) vectorized for Intel AVX-512, AMX, and Xe GPU architectures.

Question 04

How does oneDNN operator fusion (e.g., fusing Conv2D + BiasAdd + ReLU) achieve substantial performance speedups during neural network execution?

- **(a)** By increasing the total number of floating-point operations executed.
- **(b) [CORRECT]** By retaining intermediate activation tensors inside fast L1/L2 CPU caches or registers, eliminating redundant write-backs and reads from high-latency main memory (DRAM).
- **(c)** By compressing neural network weights to 1-bit representations automatically.
- **(d)** By converting all multi-threaded operations into single-threaded serial loops.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Without fusion, Conv2D writes output to DRAM, BiasAdd reads from DRAM and writes back, and ReLU reads and writes again. Operator fusion chains these operations in per-core registers/L1 cache in a single pass, slashing memory traffic and latency.

Question 05

Which environment variable explicitly forces modern TensorFlow builds to enable oneDNN graph optimizations and custom execution kernels on Intel CPUs?

- **(a)** OMP_NUM_THREADS=1
- **(b) [CORRECT]** TF_ENABLE_ONEDNN_OPTS=1
- **(c)** INTEL_MKL_DISABLE=1
- **(d)** SLURM_NNODES=4

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Application`
> **Architectural Rationale**: Setting 'TF_ENABLE_ONEDNN_OPTS=1' enables oneDNN primitive optimizations in TensorFlow, utilizing AVX-512 vectorization, AMX systolic tiles, and cache-friendly blocked tensor memory layouts.

Question 06

Why do Intel oneDNN primitives convert standard tensor layouts (like NHWC or NCHW) into blocked formats such as 'nChw16c' or 'nChw8c' internally on x86_64 processors?

- **(a)** To ensure compatibility with 32-bit legacy x86 machines.
- **(b) [CORRECT]** To align the channel dimension with hardware SIMD register widths (e.g., 512-bit registers holding 16 single-precision floats) for contiguous vector load/store instructions.
- **(c)** To encrypt the tensor data in system RAM.
- **(d)** To force all matrix computations to run on the GPU instead of the CPU.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Blocked formats chunk channels into multiples of vector register size (e.g., 16 float32 elements in 512-bit AVX-512 registers), avoiding strided memory accesses and maximizing SIMD throughput.

Question 07

What is Intel AMX (Advanced Matrix Extensions) introduced in 4th Gen Intel Xeon Scalable processors (Sapphire Rapids), and how does it accelerate deep learning?

- **(a)** A software-only Python module for matrix transposition.
- **(b) [CORRECT]** A hardware silicon architecture featuring 2D register files (Tiles) and a Tile Matrix Multiply (TMUL) accelerator designed for high-throughput BF16 and INT8 matrix operations.
- **(c)** An optical network interconnect card operating at 800 Gbps.
- **(d)** A PCIe expansion bridge for legacy VGA displays.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Intel AMX introduces hardware 2D tile registers (TMM0-TMM7) and TMUL execution units that perform up to 2048 INT8 or 1024 BF16 operations per cycle per core, dramatically accelerating DL inference and training directly on Xeon CPUs.

Question 08

What architectural capability characterizes the Intel Data Center GPU Max Series (formerly code-named Ponte Vecchio) for HPC and Deep Learning?

- **(a)** It utilizes ARM architecture with integrated SATA controllers.
- **(b) [CORRECT]** It features Xe-HPC architecture with Xe Core vector and matrix (XMX) engines, massive HBM2e memory bandwidth (up to 3.28 TB/s), and Xe Link inter-GPU fabrics.
- **(c)** It is an FPGA designed exclusively for software network emulation.
- **(d)** It requires software emulation to process standard FP32 floating point data.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: The Intel Data Center GPU Max Series incorporates Xe-HPC cores with hardware XMX (Xe Matrix eXtensions), multi-tile Foveros 3D packaging, high-bandwidth HBM2e memory, and bi-directional Xe Link bridges for multi-GPU scaling.

Question 09

Which software package enables standard TensorFlow applications to transparently target Intel Data Center GPUs (XPUs) via oneAPI SYCL runtimes?

- **(a) [CORRECT]** Intel Extension for TensorFlow (ITEX)
- **(b)** Intel MPI Benchmarks (IMB)
- **(c)** Gnuplot 5.4
- **(d)** Slurm Job Scheduler

> **Correct Answer**: **(A)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: Intel Extension for TensorFlow (ITEX) is a pluggable device library that integrates with standard TensorFlow to run graph operations and custom kernels on Intel CPUs and GPUs (XPUs) via oneAPI Level Zero and SYCL.

Question 10

What is the primary compute architecture of Habana Gaudi deep learning processors developed by Intel?

- **(a)** A standard x86 CPU with hyperthreading.
- **(b) [CORRECT]** A heterogeneous architecture combining VLIW SIMD Tensor Processor Cores (TPCs) with dedicated GEMM Matrix Multiplication Engines (MMEs).
- **(c)** A RISC-V microcontroller cluster running at 50 MHz.
- **(d)** A fixed-function ASIC strictly limited to linear regression.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Habana Gaudi processors combine a dedicated Matrix Multiplication Engine (MME) optimized for heavy GEMM operations with fully programmable VLIW SIMD Tensor Processor Cores (TPCs) for non-linear activations and custom ops.

Question 11

What unique networking integration distinguishes Habana Gaudi processors (e.g., Gaudi 2 / Gaudi 3) from conventional discrete GPUs?

- **(a)** They rely solely on external USB 3.0 dongles for inter-node communication.
- **(b) [CORRECT]** They integrate 24 standard 100GbE RoCE (RDMA over Converged Ethernet) ports directly onto each OAM processor silicon substrate, enabling all-to-all scaling without proprietary NICs.
- **(c)** They transmit training data over analog audio cables.

- **(d)** They require dedicated Ethernet switches inside every individual PCIe slot.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Habana Gaudi integrates native RDMA over Converged Ethernet (RoCE v2) directly on-die (24x 100GbE per chip on Gaudi 2), eliminating expensive external host NICs and lowering distributed scale-out networking costs.

Question 12

What is the role of Intel Habana SynapseAI software suite in distributed deep learning?

- **(a)** It is a desktop video editing utility.
- **(b) [CORRECT]** It provides the graph compiler, runtime engine, communication libraries (HCL), and framework plugins for PyTorch and TensorFlow targeting Gaudi processors.
- **(c)** It is an unmanaged hypervisor for KVM virtual machines.
- **(d)** It replaces standard Linux kernel system calls with DOS commands.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: SynapseAI is the software stack for Habana Gaudi. It includes the SynapseAI graph compiler, TPC kernel library, Habana Communication Library (HCL), and PyTorch/TensorFlow plugins (`habana_frameworks`).

Question 13

In a Slurm-managed HPC cluster, what is the role of the 'slurmctld' daemon?

- **(a)** It executes tasks directly on worker compute node CPU cores.
- **(b) [CORRECT]** It is the central management daemon that monitors cluster resources, manages job queues, and allocates nodes to pending jobs.
- **(c)** It compiles C and Fortran code into machine binaries.
- **(d)** It formats hard drives when a job completes.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: In Slurm architecture, 'slurmctld' is the central controller daemon running on the head node. It orchestrates job scheduling, evaluates resource requests, and directs node execution via per-node 'slurmd' daemons.

Question 14

Which Slurm batch script directive specifies that a distributed deep learning job requires exactly 4 physical nodes?

- **(a)** #SBATCH --time=04:00:00
- **(b) [CORRECT]** #SBATCH --nodes=4
- **(c)** #SBATCH --partition=debug
- **(d)** #SBATCH --mem=4G

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Application`
> **Architectural Rationale**: The directive '#SBATCH --nodes=4' (or '-N 4') requests exactly 4 distinct compute nodes from the cluster partition for the batch allocation.

Question 15

What does the Slurm command 'squeue' perform when executed by an HPC engineer?

- **(a)** It deletes all user files in the /tmp partition.
- **(b) [CORRECT]** It queries and displays the status of jobs currently in the scheduling queue (running, pending, or completing).
- **(c)** It reboots all compute nodes in the cluster immediately.
- **(d)** It prints the temperature of the server room air conditioning.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: 'squeue' inspects the state of jobs queued in Slurm, displaying JobID, Partition, Job Name, User, State (R=Running, PD=Pending), Time Used, and Nodes Allocated.

Question 16

Which environment variable is automatically injected by Slurm into a running job script to indicate the total number of allocated compute nodes?

- **(a) [CORRECT]** SLURM_NNODES
- **(b)** OMP_NUM_THREADS
- **(c)** CUDA_VISIBLE_DEVICES
- **(d)** PYTHONPATH

> **Correct Answer**: **(A)**
> **Bloom's Cognitive Level**: `Application`
> **Architectural Rationale**: Slurm exports 'SLURM_NNODES' (or 'SLURM_JOB_NUM_NODES') containing the total count of compute nodes allocated to the job, crucial for configuring multi-node distributed training scripts.

Question 17

Why is the '#SBATCH --exclusive' flag frequently used when submitting multi-node deep learning benchmark jobs to Slurm?

- **(a)** To prevent anyone from reading the final research paper.
- **(b) [CORRECT]** To ensure allocated nodes are dedicated 100% to the job, preventing CPU, RAM, and PCIe bandwidth contention from other tenants' noisy-neighbor jobs.
- **(c)** To run the job exclusively on the master head node.
- **(d)** To permanently reserve the nodes even after the job terminates.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: '--exclusive' grants full access to all node resources, guaranteeing that background processes or competing workloads do not introduce non-deterministic latency or cache pollution into DL benchmarks.

Question 18

How does Horovod differ fundamentally from traditional Parameter Server (PS) architectures in distributed deep learning?

- **(a)** Horovod requires a dedicated central master server that stores all parameters and collects all gradients.
- **(b) [CORRECT]** Horovod utilizes a decentralized Ring-AllReduce algorithm over MPI/NCCL, where all workers communicate peer-to-peer and compute aggregate gradients collaboratively.
- **(c)** Horovod only runs on single-core mobile processors.
- **(d)** Horovod discards gradients instead of averaging them across workers.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: In Parameter Server architectures, the central PS server becomes a network bandwidth bottleneck as worker counts grow. Horovod replaces this with Ring-AllReduce, sharing communication load equally across all nodes.

Question 19

What is the primary network scalability bottleneck of the Parameter Server (PS) architecture when scaling deep learning training to 64+ nodes?

- **(a)** The PS server runs out of disk storage for log files.
- **(b) [CORRECT]** The network bandwidth on the parameter server(s) scales linearly with the number of workers ($O(N)$), causing severe network saturation and idle worker stall.
- **(c)** Python garbage collection stops all network switches.
- **(d)** Ethernet packets cannot travel across more than 2 switches.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Because every worker transmits gradients to and receives updated weights from the parameter servers, network traffic at the parameter servers increases with N, creating an insurmountable bottleneck at scale.

Question 20

In the Ring-AllReduce algorithm utilized by Horovod, what is the total data volume transferred by each worker node to complete a full gradient synchronization of size M bytes across N workers?

- **(a) [CORRECT]** $2 \times \frac{N-1}{N} \times M$
- **(b)** $N^2 \times M$
- **(c)** $\frac{M}{N^2}$
- **(d)** $M \times \log_2(N) \times N$

Correct Answer: **(A)**
Bloom's Cognitive Level: `Apply`
Architectural Rationale: Ring-AllReduce consists of two phases: Scatter-Reduce ($(N-1)$ steps of size M/N) and AllGather ($(N-1)$ steps of size M/N). Total data sent per node is $2 \times \frac{N-1}{N} \times M$, which asymptotically approaches $2M$ as N grows, making bandwidth per worker completely independent of the number of nodes!

Question 21

Which Horovod API function must be invoked at the very beginning of a distributed training script to initialize the communication runtime?

- **(a)** hvd.shutdown()
- **(b) [CORRECT]** hvd.init()
- **(c)** hvd.broadcast_variables()
- **(d)** hvd.rank()

Correct Answer: **(B)**
Bloom's Cognitive Level: `Knowledge`
Architectural Rationale: 'hvd.init()' initializes the Horovod runtime, sets up underlying MPI/Gloo communicator communicators, and registers world size and rank IDs.

Question 22

In a multi-GPU/XPU server running Horovod, why is it critical to bind each rank using 'hvd.local_rank()' rather than 'hvd.rank()'?

- **(a)** 'hvd.rank()' returns negative numbers on Linux.
- **(b) [CORRECT]** 'hvd.local_rank()' provides the device index local to the physical node (e.g., 0 to 7), preventing multiple processes on the same host from contending for GPU 0.
- **(c)** 'hvd.local_rank()' disables GPU acceleration to save power.
- **(d)** 'hvd.local_rank()' causes all workers to run on CPU core 0.

Correct Answer: **(B)**
Bloom's Cognitive Level: `Application`
Architectural Rationale: On a cluster of 4 nodes with 4 GPUs each (16 total), global ranks run 0-15. Using global rank 5 on Node 1 would crash because Node 1 only has GPUs 0-3. 'hvd.local_rank()' returns 0-3 on each node, correctly mapping processes to local accelerators.

Question 23

In TensorFlow 2.x with Horovod, how is automatic gradient averaging across all distributed workers implemented?

- **(a)** By manually writing an MPI_Allreduce loop inside a C++ extension.
- **(b) [CORRECT]** By wrapping the standard TensorFlow optimizer with 'hvd.DistributedOptimizer(optimizer)'.
- **(c)** By setting learning rate to zero.
- **(d)** By saving weights to disk after every batch and reloading them.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Application`
> **Architectural Rationale**: 'hvd.DistributedOptimizer' wraps an existing Keras/TF optimizer (e.g., Adam, SGD) and intercepts the gradient computation, injecting an allreduce operation to average gradients across all ranks before applying weight updates.

Question 24

Why is 'hvd.broadcast_variables(model.variables, root_rank=0)' required before commencing the training loop in Horovod?

- **(a)** To print the neural network architecture to the console.
- **(b) [CORRECT]** To ensure all worker ranks initialize with identical initial model weights and optimizer states from root rank 0, preventing divergent optimization paths.
- **(c)** To delete random weights from memory.
- **(d)** To upload model checkpoints to a public cloud bucket.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Neural networks initialize weights using random seeds. Unless rank 0 broadcasts its initial parameters to all workers, each worker starts with different weights, leading to corrupted, non-converging gradient averaging.

Question 25

According to Goyal et al.'s Linear Scaling Rule in distributed deep learning, how should the base learning rate (LR_{base}) typically be adjusted when scaling batch size from a single worker to N workers?

- **(a)** $LR_{new} = LR_{base} / N$
- **(b) [CORRECT]** $LR_{new} = LR_{base} \times N$
- **(c)** $LR_{new} = LR_{base} \times \sqrt{N}$
- **(d)** $LR_{new} = LR_{base} / \log(N)$

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: When effective batch size is scaled by N (minibatch B per worker across N workers = $N \times B$), the Linear Scaling Rule states that the learning rate should be multiplied by N ($LR_{new} = N \times LR_{base}$) to match the expected weight update magnitude per step.
